ApexVerify

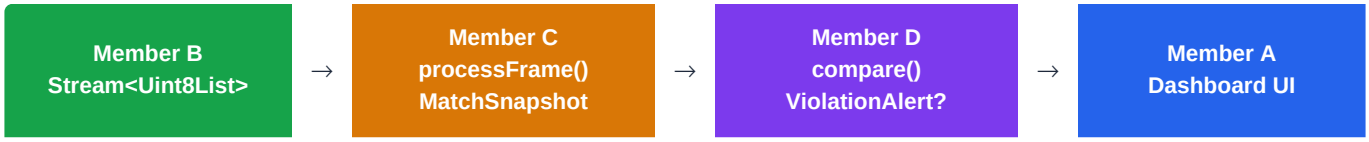
Integration Day — Execution Guide

Google Solution Challenge 2026

All four members are present. All individual work is complete. This guide takes you through Phase 3 — wiring everything together — step by step, in the correct order, with exact instructions for every member at every stage.

Member A	Member B	Member C	Member D
Frontend / UI	Frame Pipeline	OCR / Refactoring	Backend / Firebase

Pipeline at a glance



Phase 3 — Steps at a glance

Step	Who	What	Expected Duration
3.1	B + C	Connect real frame stream to OCR engine	30–45 min
3.2	C + D	Feed MatchSnapshot into comparison + fire alerts	30–45 min
3.3	D + A	Wire alert stream to dashboard, swap mock → real	30–45 min
3.4	All 4	Full end-to-end test + screen recording for submission	30–60 min

Pre-Flight Checks — Before You Touch Any Code

Every member must confirm the items below before Step 3.1 begins. Do not start integration until all four columns are ticked.

Member A ✓	Member B ✓	Member C ✓	Member D ✓
UI shell runs with MockFrameSampler	FrameSampler.startSampling() returns Stream<List…	OcrService.processFrame() works on static PNGs	Firebase project live, config in repo
Alert card widget built	Grayscale + contrast (1.5) preprocessing applied	ML Kit path tested and returning text	Firestore seeded: matches/{id}/official with ho
DMCA log panel built	MockFrameSampler committed to repo	Cloud Vision escalation tested	ComparisonService.compare() tested with ma
Status indicator widget built	Test PNGs saved to disk and verified	Regex parser populates all MatchSnapshot fields	Gemini Flash call tested and returning descrip
Screenshot button (stub) wired	Real YouTube / Twitch URL ready to paste	GCP service account key is not in the repo	Stream<ViolationAlert?> compiles and emits

■ **DO NOT PROCEED UNT** **All four members must have their Part 1 work committed and pushed to the shared repo before Ste**

Step 3.1 — Connect Frame Pipeline to OCR Engine (Member B + Member

Members A and D are idle during this step — they should review their own code or rest. Only B and C are active. Expected duration: 30–45 minutes.

Member B — Provide the real stream

1 Pull latest from shared repo and confirm your branch is clean

```
git pull origin main
git status # should be clean
```

2 Confirm FrameSampler compiles with no errors

Run the project and confirm no build errors. Member C will subscribe to your stream, so it must compile cleanly before they start.

3 Identify a real stream URL to use for testing

Use a real YouTube live stream URL or a Twitch stream URL. Sports channels with active scoreboards are best — Premier League, NBA, or any league currently in season. Paste the URL into a shared note or chat so Member C can copy it exactly.

```
// Example — paste your real URL here:
const testUrl = 'https://www.youtube.com/watch?v=XXXXXXXXXXX';
```

4 Confirm your stream outputs correctly — run your own test subscriber first

Before handing to Member C, run your own PNG save test to confirm the stream is producing frames:

```
frameSampler.startSampling(testUrl).listen((Uint8List frameBytes) {
final file = File('verify_${DateTime.now().millisecondsSinceEpoch}.png');
file.writeAsBytesSync(frameBytes);
print('Frame saved: ${file.path}');
});
```

Open the saved PNG. Confirm it is grayscale and that scoreboard text looks crisp. If the frame looks blurry or the text is washed out, increase contrast value from 1.5 to 1.8 before calling Member C over.

5 Signal Member C — stream is ready

Tell Member C: the URL, that FrameSampler is running, and the expected output format (Uint8List, one emission every 5 seconds, grayscale + contrast processed).

Member C — Subscribe to the stream

Wait for Member B's signal before starting any of the steps below.

1 Pull latest from shared repo

```
git pull origin main
```

2 Replace the static file input with Member B's live stream

In ocr_service.dart or your test runner, change from the Part 1 static approach to the live subscription:

```
// PART 1 (remove this):
final bytes = await File('test_scoreboard.png').readAsBytes();
final snapshot = await ocrService.processFrame(bytes);

// PART 2 (add this):
frameSampler.startSampling(url).listen((Uint8List frameBytes) async {
final snapshot = await ocrService.processFrame(frameBytes);
print('Score: ${snapshot.score}');
print('Clock: ${snapshot.clock}');
print('Overlay: ${snapshot.hasOverlay}');
_snapshotController.add(snapshot); // emit to Member D
});
```

3 Confirm MatchSnapshot values are populating from real broadcast frames

Watch the console. Every 5 seconds a new MatchSnapshot should print. Both score and clock should contain real values — not empty strings.

If score and clock are empty strings — your Regex needs tuning. Common adjustments for live broadcasts:

```
// Score — broadcast compression can add spaces; be more flexible:
RegExp(r'\b(\d{1,2})\s*[-:–]\s*(\d{1,2})\b')

// Clock — try a broader match if yours isn't catching it:
RegExp(r"\b(\d{1,3}(?:[+:]\d+)?)['\"\s]|\b\d{1,2}:\d{2}\b"")
```

If ML Kit returns empty text on real broadcast frames (even though it worked on static PNGs), ask Member B to increase contrast to 1.8 — broadcast compression artifacts can reduce text sharpness.

4 Confirm Cloud Vision escalation triggers correctly

Wait for a low-quality frame (you will see it in the logs when ML Kit returns fewer than 10 characters). Confirm the log shows the Cloud Vision API being called and returning a longer string.

5 Confirm the snapshotStream is exposed and broadcasting

```
final _snapshotController = StreamController.broadcast();
Stream get snapshotStream => _snapshotController.stream;
```

Tell Member D that the snapshotStream is ready and subscribable. This is the trigger for Step 3.2.

✓ **SUCCESS CONDITION** **Member C's console prints a populated MatchSnapshot every ~5 seconds. score and clock fields co**

Step 3.2 — Connect OCR Output to Comparison Engine (Member C + Mem

Members A and B are idle during this step. Member B can monitor their stream logs to ensure the frame pipeline keeps running. Only C and D are active. Expected duration: 30–45 minutes.

Member C — Pass snapshotStream to Member D

1 Subscribe Member D's ComparisonService to your snapshotStream

Pass the live stream into Member D's comparison service:

```
ocrService.snapshotStream.listen((MatchSnapshot snap) {
comparisonService.compare(snap);
});
```

2 Deliberately corrupt a Firestore value to test alert firing

Open the Firebase console. Navigate to matches/{matchId}/official. Change the score field to something wrong — for example, if the real score is '2 - 1', change it to '9 - 9'. This simulates a stream manipulation.

Watch Member D's console. A ViolationAlert should appear within 10 seconds of the next frame emission.

3 Restore the correct Firestore value after the test

Set the score field back to the correct real value immediately after confirming the alert fired.

Member D — Run comparison and fire alerts

Wait for Member C to pass the snapshotStream before starting these steps.

1 Subscribe ComparisonService to Member C's snapshotStream

```
ocrService.snapshotStream.listen((MatchSnapshot snap) async {
final alert = await comparisonService.compare(snap);
_alertController.add(alert); // null = clean, non-null = violation
});
```

2 Confirm Firestore fetch is working against the real seeded match

Log the official data being fetched from Firestore on each comparison call. Confirm it is pulling homeTeam, awayTeam, score, and clock from the correct document.

```
// Add this temporarily for debugging during Step 3.2:
print('Official from Firestore: $officialScore vs OCR: ${snap.score}');
```

3 Confirm ViolationAlert is created when Member C corrupts the Firestore value

When the score mismatch is detected, verify the ViolationAlert fields are all populated:

```
print('severity: ${alert.severity}'); // should be HIGH or similar
```

```
print('fieldMismatch: ${alert.fieldMismatch}'); // should be 'score'
print('expected: ${alert.expected}'); // Firestore value
print('actual: ${alert.actual}'); // OCR value
print('description: ${alert.description}'); // Gemini Flash text — must not be null
print('timestamp: ${alert.timestamp}');
```

The Gemini description field is critical. If it is null, your Gemini Flash call is failing. Check: (1) API key is correct, (2) frame bytes are being passed, (3) the model string is 'gemini-1.5-flash'.

4 Confirm clean frames produce null alerts

After restoring the correct Firestore score, watch your console for 2–3 more frames. Each should log a null alert — confirming the clean path works correctly.

5 Expose Stream<ViolationAlert?> to Member A

```
final _alertController = StreamController.broadcast();
Stream get alertStream => _alertController.stream;
```

Tell Member A that alertStream is ready and subscribable. This is the trigger for Step 3.3.

✓ **SUCCESS CONDITION** **A deliberate Firestore mismatch produces a ViolationAlert with a non-null Gemini description within**

Step 3.3 — Wire Alert Stream to Dashboard (Member D + Member A)

Members B and C are idle — they should keep their pipeline running so the live stream continues feeding the system. Only D and A are active. Expected duration: 30–45 minutes.

Member D — Expose alert stream to Member A's dashboard

1 Confirm Member A can import and subscribe to alertStream without compile errors

Sit next to Member A. Watch them import your ComparisonService into the dashboard. Confirm there are no import path issues. The stream type must be exactly Stream.

2 Trigger a test violation while Member A's dashboard is open

After Member A has the StreamBuilder wired (see Member A steps below), corrupt the Firestore score again. Watch Member A's screen together — the status indicator should turn red within 10 seconds.

3 Restore correct Firestore value and confirm green state returns

Set score back to the correct value. The next clean frame should emit a null alert. Member A's dashboard should return to green on the following 5-second cycle.

Member A — Wire real data, complete Part 2

Wait for Member D's signal that alertStream is ready before making any changes.

1 Swap MockFrameSampler for real FrameSampler — one line change

```
// Part 1 — remove this line:
final sampler = MockFrameSampler();

// Part 2 — replace with this single line:
final sampler = FrameSampler();
```

Nothing else in your code needs to change. The Stream interface is identical.

2 Wire the frame preview panel to the real stream

The frame preview StreamBuilder already exists from Part 1. Confirm it is still working after the sampler swap — frames should still update every 5 seconds, now sourced from the real pipeline.

3 Wire StreamBuilder to Member D's alertStream

```
StreamBuilder(
stream: comparisonService.alertStream,
builder: (context, snapshot) {
final alert = snapshot.data;
return Column(children: [
StatusIndicator(isViolation: alert != null),
```

```
if (alert != null) AlertCard(
severity: alert.severity,
description: alert.description,
timestamp: alert.timestamp,
fieldMismatch: alert.fieldMismatch,
),
]);
},
)
```

4 Make the DMCA log append on each non-null alert

```
// In your Riverpod notifier or StatefulWidget setState:
comparisonService.alertStream.listen((ViolationAlert? alert) {
if (alert != null) {
setState(() => _dmcaLog.add(alert));
}
});
```

5 Test the status indicator, alert card, and DMCA log together

Ask Member D to corrupt the Firestore score. Verify all three behaviours simultaneously:

- [ ] Status indicator turns red within 10 seconds
- [ ] Alert card shows real Gemini description text — not placeholder
- [ ] DMCA log appends the new ViolationAlert entry
- [ ] Restoring correct Firestore value returns status to green on the next clean frame

6 Test the screenshot save button end-to-end

Click the screenshot button with the real FrameSampler active. Confirm a PNG file is written to disk with a timestamped filename. Open the file and confirm it is readable.

✓ **SUCCESS CONDITION** **Dashboard turns red within 10 seconds of a deliberate Firestore mismatch. Alert card renders real G**

Step 3.4 — Full End-to-End Test (All 4 Members)

All four members are active. Run through the test script below exactly once as a team. Record the entire session as a screen recording — this is your submission demo video. Expected duration: 30–60 minutes.

Test Script

#	Action	Expected Result	Who Confirms
1	Member A pastes a real stream URL into the URL input field and clicks Start Monitoring	Frame preview panel begins updating every 5 seconds with real broadcast frames	A + B
2	All members watch console output for 2 full minutes	Member C's console shows MatchSnapshot with real score and clock values every	C
3	Member D corrupts the Firestore score field (e.g. changes 2 to 9)	Within 10 seconds of the next frame: status indicator turns red, alert card appears	D + A
4	Member D restores the correct Firestore score value	Within 10 seconds of the next clean frame: status indicator returns to green, alert c	D + A
5	Member D corrupts the clock field (e.g. changes "45:00" to "90:00")	Validation alert fires again — this time field Mismatch should read 'clock' not 'score'	D + A
6	Member D restores correct clock value	Clean frame → null alert → green status	D + A
7	Member A clicks the Screenshot Save button	PNG file written to disk with timestamped filename — Member A opens the file and	A + B
8	Member B confirms the frame pipeline has been running continuously throughout	No gaps in frame timestamps — consistent 5-second cadence maintained across th	B
9	Member C checks OCR processing time for the last 10 frames	All under 5000ms — no frame queue buildup	C

Screen Recording Requirements

The screen recording is your demo video for the Google Solution Challenge submission. It must show all 9 steps above in a single uncut recording. Ensure the following are visible:

- [ ] The dashboard UI — frame preview, status indicator, alert card, DMCA log panel
- [ ] The console output from Member C showing populated MatchSnapshot values
- [ ] The status indicator turning red when the Firestore value is corrupted
- [ ] The alert card with real Gemini-generated description text
- [ ] The DMCA log appending a new entry
- [ ] The status indicator returning to green when Firestore is restored
- [ ] The clock field mismatch test (test #5)
- [ ] The screenshot PNG written to disk (test #7)

✓ **SUCCESS CONDITION** **All 9 test actions produce their expected results. Screen recording is complete and saved. The proje**

Troubleshooting Reference

Member B — Frame Pipeline

Problem 1	Stream emits frames but they are all black or corrupted
Fix	The video_player package may not support the stream URL format. Try a different URL. YouTube live URLs sometimes require a
Problem 2	Frames look too dark after contrast boost
Fix	Reduce contrast from 1.5 to 1.2. Grayscale conversion + contrast is cumulative — overboost makes bright regions white-out, whic
Problem 3	dart:isolate errors on Windows
Fix	If Isolate.spawn() fails on Windows desktop, move the preprocessing back to the main isolate using a simple async function instea

Member C — OCR Engine

Problem 1	ML Kit throws 'InputImage metadata is required' on desktop
Fix	On Flutter desktop, InputImage.fromBytes() requires explicit metadata including image size and format. Decode the image first wit
Problem 2	score and clock are empty strings on real broadcast frames but worked on static PNGs
Fix	Real broadcast frames have JPEG compression artifacts that reduce sharpness. Ask Member B to increase contrast to 1.8. Also t
Problem 3	Cloud Vision API returns 403 Forbidden
Fix	The service account key does not have the Cloud Vision API role. Open GCP console → IAM → find the service account → add th
Problem 4	processFrame() takes more than 5 seconds
Fix	ML Kit is fast but Cloud Vision escalation adds latency. Ensure you are only calling Cloud Vision when ML Kit returns fewer than 1

Member D — Backend / Firebase

Problem 1	Gemini Flash returns null description
Fix	Check that you are passing frame bytes to the Gemini call, not an empty list. Log the byte length before the call — it should be sev
Problem 2	ComparisonService.compare() never fires ViolationAlert even with wrong Firestore value
Fix	Add a debug print showing both the OCR value and Firestore value side by side. The most common cause is a whitespace or case
Problem 3	Firestore read throws permission denied
Fix	Check Firestore rules in the Firebase console. For dev/testing, set rules to allow read/write for all authenticated users. Ensure fireb

Member A — Dashboard

Problem 1	StreamBuilder shows CircularProgressIndicator indefinitely after swapping to real FrameSampler
Fix	The real FrameSampler takes ~5 seconds to emit its first frame (unlike MockFrameSampler which emits immediately). Add a timeo
Problem 2	Status indicator stays red after Firestore value is restored

Fix	The StreamBuilder only rebuilds when a new event arrives. If Member D's alertStream emits null (clean) on the next frame, the ind
Problem 3	DMCA log is not appending entries
Fix	Confirm you are calling setState() (or notifying the Riverpod provider) inside the alertStream listener. If the listener is set up outside

Integration Complete — Final Checklist

Before declaring the integration done and the project submission-ready, every item below must be ticked by the responsible member.

Member B — Frame Pipeline

- [ ] FrameSampler running against real stream URL for the full duration of Step 3.4
- [ ] Grayscale + contrast preprocessing confirmed in saved test PNGs
- [ ] MockFrameSampler committed to repo (Member A used it in Part 1)
- [ ] Stream<Uint8List> interface unchanged — Member C subscribed with no modifications
- [ ] No isolate or threading errors on Windows desktop during the full test

Member C — OCR / Refactoring

- [ ] snapshotStream emitting populated MatchSnapshot every ~5 seconds from live stream
- [ ] score and clock fields contain real values — not empty strings
- [ ] Cloud Vision escalation confirmed triggering on low-confidence ML Kit results
- [ ] All MatchSnapshot fields match the agreed model class exactly (no field name drift)
- [ ] processFrame() processing time confirmed under 5000ms for all frames in Step 3.4

Member D — Backend / Firebase

- [ ] ComparisonService.compare() producing ViolationAlert for every Firestore mismatch in test
- [ ] Gemini Flash description is non-null in every ViolationAlert produced
- [ ] alertStream broadcasting correctly — Member A subscribed without errors
- [ ] Clean frames (after Firestore restore) produce null alerts — no false positives
- [ ] Firestore seeded data matches the stream being tested (correct homeTeam, awayTeam)

Member A — Frontend / UI

- [ ] Frame preview updating every 5 seconds with real broadcast frames
- [ ] Status indicator turns red within 10 seconds of Firestore corruption
- [ ] Alert card renders real Gemini description text
- [ ] DMCA log appends each ViolationAlert correctly
- [ ] Status returns to green after Firestore restore on next clean frame
- [ ] Screenshot button writes timestamped PNG to disk
- [ ] Screen recording of Step 3.4 saved — all 9 test actions captured

ApexVerify — Google Solution Challenge 2026 — Good luck!